Defiende el servidor: versionado de una solución de ML para UNSW-NB15

Francy Carolina Umbarila Rodriguez · Grupo A · entrega individual · MLOps, Visualización 2, 2026-II · Universidad Externado de Colombia
Código: `github.com/c851458/Taller_visu_2` · Modelos: `huggingface.co/c851458/defiende-servidor-unsw` · DVC: `dagshub.com/c851458/Taller_visu_2.dvc`

El objetivo es clasificar conexiones como normales (0) o ataques (1, clase positiva) y, sobre todo, poder identificar, comparar y recuperar cada versión de la solución. La cadena de trazabilidad es **commit Git → referencia DVC → run de MLflow → revisión (SHA) de Hugging Face**; cada eslabón se vincula por identificadores y hashes, no por nombres de archivo. Todas las cifras de este informe se leen de los archivos de evidencias/ (este PDF se genera con scripts/generar_informe.py).

1. Datos, auditoría y limpieza

Se usan las particiones publicadas, descargadas de la copia alternativa indicada en el taller (mirror de terceros); la atribución corresponde a la página oficial de UNSW. Las huellas SHA-256 de ambos CSV coinciden con las del anexo del taller. Ningún archivo tiene valores faltantes ni valores negativos en las columnas numéricas, y los id son únicos. Se excluyen de las entradas label (respuesta), attack_cat (revela la respuesta: Normal ⇔ label 0) e id (identificador). Quedan 42 variables predictoras, 3 de ellas categóricas (proto, service, state).

Archivo	SHA-256 (inicio)	Filas × col.	Normal / ataque	Filas en grupos duplicados	Grupos contradictorios
training-set	`bec7dd5ec88dc2a0…`	175.341 × 45	56.000 / 119.341	84.049 (74.301 redundantes)	229 grupos, 940 filas
testing-set	`734fe6642edf758f…`	82.332 × 45	37.000 / 45.332	34.704 (28.386 redundantes)	6 grupos, 14 filas

Duplicados y contradicciones. Un grupo es un conjunto de filas con todas las variables predictoras idénticas (sin id ni etiquetas). Política: (1) en el archivo de entrenamiento se eliminan las 940 filas de los 229 grupos con etiquetas contradictorias, porque no son aprendibles; (2) los duplicados exactos se conservan, pero cada grupo va **completo** a una sola partición; (3) en test no se elimina ninguna fila, solo se audita.

Particiones. Entrenamiento y validación salen solo del archivo de entrenamiento, con una asignación determinista por grupo: sha256("2026:group_id") < 0,20. Resultado: train = 139.166, validación = 35.235 y **0 grupos compartidos** entre ambas. Las huellas de cada split y la asignación fila → partición (splits.csv) están en auditoria.json. Las cuatro configuraciones usan exactamente estas particiones, y codificadores, imputación y escalado se ajustan dentro de un Pipeline de scikit-learn solo con train. Test = archivo completo (82.332 filas). **Solapamiento:** 8.313 filas de test (10,1 %) tienen entradas idénticas a alguna fila de train/val, así que el test no es del todo "datos nuevos".

Dos versiones de datos con DVC. data/raw.dvc (CSV recibidos + procedencia, md5 `688fd73cceff08e318906ecd85de60c4.dir`, tag Git datos-original) y la etapa prepare de dvc.yaml / dvc.lock (train/val/test/splits, md5 `5832cbec6fbaff34eaf128c733c18932.dir`, tag datos-preparados). El almacenamiento está en DagsHub, no en el computador local.

2. Cuatro experimentos, dos familias (validación)

Cada configuración (configs/experiments/*.yaml) declara su hipótesis, el cambio respecto a la anterior y la política de umbral. Cada run de MLflow registra parámetros, semilla (2026), commit, md5 DVC, métricas de validación, matriz de confusión y el modelo completo (preprocesamiento + estimador). La exportación está en evidencias/mlflow/ (parámetros, métricas, tags y un manifiesto con el SHA-256 de cada artefacto; los modelos de E1, E2 y E4 van copiados) y la tabla en experimentos.csv. **Denominadores cero:** si un cociente es indefinido (p. ej., precision sin positivos predichos) se reporta 0,0 y la métrica se lista en undefined_metrics; en estos runs no ocurrió.

Experimento	Familia	Hipótesis	Run ID	Umbral	Recall	Precision	FPR	Macro-F1	FP	FN
E1 · LogReg base	lineal	Referencia: one-hot + escalado, umbral 0,5.	`52c52259`	0,5000	0,989	0,928	0,170	0,926	1.864	273
E2 · LogReg + log1p	lineal	log1p en variables de cola larga reduce FP.	`2a5fc84c`	0,5000	0,977	0,942	0,133	0,932	1.458	549
E3 · Random Forest	árboles	Árboles capturan interacciones no lineales.	`c5368394`	0,5000	0,981	0,958	0,097	0,949	1.057	469
E4 · HistGB + umbral	árboles	Boosting + umbral elegido con FPR ≤ 5 %.	`c5503f3f`	0,6574	0,960	0,977	0,050	0,951	547	960

Interpretación. E1 detecta casi todos los ataques, pero 1 de cada 6 conexiones normales genera una falsa alarma. E2 confirma la hipótesis solo en parte: log1p reduce la FPR (0,170 → 0,133) a costa de algo de recall. El cambio de familia (E3) es lo que más ayuda: macro-F1 sube a 0,949 y la FPR baja a menos del 10 % con el mismo umbral. E4 consigue la mejor precision y el mejor macro-F1 al fijar explícitamente la tasa de falsas alarmas. Con

umbral 0,5, E4 tendría un macro-F1 algo mayor (0,953) pero una FPR de 0,080: el umbral cambia la proporción entre los dos tipos de error, no la calidad del ranking. Validación: 10.945 normales y 24.290 ataques.

Categoría (attack_cat)	Filas val	E1	E2	E3	E4
Normal (falsos positivos)	10.945	1.864 (17,0 %)	1.458 (13,3 %)	1.057 (9,7 %)	547 (5,0 %)
Fuzzers (no detectados)	3.547	177 (5,0 %)	467 (13,2 %)	402 (11,3 %)	825 (23,3 %)
Analysis (no detectados)	421	0 (0,0 %)	16 (3,8 %)	41 (9,7 %)	59 (14,0 %)
Exploits (no detectados)	6.852	54 (0,8 %)	28 (0,4 %)	18 (0,3 %)	52 (0,8 %)
DoS (no detectados)	2.606	21 (0,8 %)	18 (0,7 %)	2 (0,1 %)	9 (0,3 %)
Reconnaissance (no detectados)	1.926	7 (0,4 %)	11 (0,6 %)	4 (0,2 %)	9 (0,5 %)
Backdoor (no detectados)	372	9 (2,4 %)	3 (0,8 %)	0 (0,0 %)	0 (0,0 %)
Generic + Shellcode + Worms (no detectados)	8.566	5	6	2	6

Errores por categoría. src/analyze_val_errors.py recarga cada modelo (hash verificado), predice validación, comprueba que las cuatro matrices de confusión coinciden con MLflow y desglosa los errores por attack_cat. attack_cat no es entrada del modelo: aquí solo sirve para explicar los errores (errores-validacion.csv/.json). El análisis es posterior a la selección y no cambió modelo ni umbral. **Fuzzers** concentra los ataques no detectados en todas las configuraciones: 65 % de los de E1 y 86 % de los de E4. Son ataques que envían datos generados al azar, y el modelo los confunde con tráfico normal. **Analysis** también empeora con los árboles (0 → 59 de 421). Generic, DoS, Exploits y Reconnaissance se detectan en más del 99 % con cualquier modelo. Las falsas alarmas se concentran en conexiones normales con sttl = 254: 88 % de las FP de E1 y 92 % de las de E4.

3. Versiones v1 y v2

	v1	v2
Experimento / familia	E1 · regresión logística	E4 · HistGradientBoosting
Preprocesamiento	One-hot (min_frequency 20) + escalado estándar	One-hot (min_frequency 20), sin escalado
Umbral (y_pred = 1 si score ≥ umbral)	0,5000	0,6574
Commit que entrenó / tag Git	`bec239b75d48` / v1	`057957069ef9` / v2
Run de MLflow	`52c5225915704ba3bbe1c91635ea6846`	`c5503f3f886f4309927f98bac0288479`
Revisión Hugging Face	`ca61db0e29f5c1816f68790f44d6af65a23ab082`	`9fb15e97dc936b6ba90bbdc73a8de92781462f75`
sha256 de model.joblib	`cd0d72260f75535658480f24ab1b3b2a…`	`689fb002a0eaa577af78ee78cec88043…`
Datos DVC (raw / preparados)	`688fd73cce…` / `5832cbec6f…`	`688fd73cce…` / `5832cbec6f…`
Validación: recall / precision	0,989 / 0,928	0,960 / 0,977
Validación: FPR / macro-F1	0,170 / 0,926	0,050 / 0,951
Validación: FP / FN	1.864 / 273	547 / 960

Qué cambia y por qué. v2 cambia la familia del modelo y la regla del umbral. Mejora la precision y el macro-F1 y reduce las falsas alarmas de 1.864 a 547 (menos de un tercio). **Empeora el recall de ataque** (0,989 → 0,960): los ataques no detectados pasan de 273 a 960, y casi todo el aumento son Fuzzers (177 → 825). Ese es el error que se acepta, porque para un equipo de seguridad el exceso de alertas provoca fatiga y hace que se ignoren también las alertas reales. Exploits, DoS, Backdoor y Shellcode siguen detectándose en ≥ 98 %. v2 es un artefacto distinto (otro hash y otra revisión), no un renombre de v1. Cada revisión de Hugging Face contiene el modelo completo, config.json con el umbral, requirements.txt, predict.py y una model card con propósito, datos, métricas, limitaciones, uso y las referencias a Git, DVC y MLflow.

4. Elección de umbral y congelación de la selección

La regla (configs/seleccion.yaml) se escribió antes de mirar el test: **mayor macro-F1 de validación entre los experimentos con FPR <= max_fpr**, con FPR máxima de 5 %. En E4 el umbral es el que da el máximo recall de validación con FPR ≤ 5 %: 0,6574. Solo E4 cumple la restricción, así que se elige como v2 (validación: FPR 0,050, macro-F1 0,951). seleccion.json (run, revisión HF, hash del modelo, umbral, razón y test_used_for_selection = false) se congeló en el commit `269c2ab0850129` (tag v2, 2026-10-28 14:38:24+00:00 UTC), anterior al commit del test, `d71d7f66addf23`. evaluate_test se niega a correr si seleccion.json no está en Git o si el test ya se evaluó. Limitación: el umbral se elige y se reporta sobre la misma validación, así que esa estimación es algo optimista.

5. Evaluación final en test (una sola vez)

v2 congelada (revisión `9fb15e97dc93`, sha256 `689fb002a0ea`…) se evaluó una vez sobre las 82.332 filas del test (sha256 del archivo preparado `23db4edbf75b`…; del CSV original `734fe6642edf`…). predicciones-test.csv tiene una fila por registro (row_id = test:<fila>, y_true, score_attack = probabilidad de ataque, y_pred), y recompute_metrics recalcula desde ese archivo exactamente las mismas métricas (diferencia 0).

Métrica	Validación	Test		Matriz de confusión (test)	Pred. normal	Pred. ataque
Recall de ataque	0,960	0,968		Real normal	29.895	7.105
Precision de ataque	0,977	0,861		Real ataque	1.459	43.873
FPR	0,050	**0,192**				
Macro-F1	0,951	0,893		Accuracy (complementaria)		0,896

El resultado empeora y se conserva sin ajustes. El recall de ataque se mantiene (0,968), pero la FPR sube de 0,050 a 0,192: 7.105 de 37.000 conexiones normales generan alarma. No se cambió el modelo ni el umbral. Para entenderlo se hizo un **diagnóstico posterior** (src/analyze_fp_test.py → analisis-fp-test.json), que solo explica el resultado y no se usó para ninguna decisión:

Segmento de conexiones normales	% de normales (val → test)	FPR val	FPR test	% de las FP de test
sttl = 254	19,4 % → 40,7 %	0,238	0,414	87,6 %
sttl = 62	3,9 % → 11,2 %	0,095	0,213	12,4 %
sttl = 31	72,5 % → 45,1 %	0,000	0,000	0,0 %
state = INT	8,3 % → 12,1 %	0,124	0,547	34,5 %
state = FIN	67,8 % → 65,3 %	0,058	0,192	65,2 %
state = CON	22,3 % → 17,9 %	0,000	0,001	0,1 %
service = "-" (sin servicio identificado)	64,1 % → 74,0 %	0,064	0,215	82,8 %
service = http	10,0 % → 10,8 %	0,063	0,252	14,2 %
service = dns	14,1 % → 8,3 %	0,000	0,000	0,0 %

El 87,6 % de las falsas alarmas vienen de conexiones normales con sttl (TTL de origen) = 254. En el archivo de entrenamiento, el 90,4 % de las filas con sttl = 254 son ataques y solo el 19,5 % del tráfico normal tiene ese valor, así que el modelo aprendió que "sttl = 254 ⇒ ataque". En test, en cambio, el 40,7 % del tráfico normal tiene sttl = 254 (y solo el 72,2 % de esas filas son ataques). Además, ese tráfico es más difícil: incluso dentro del segmento, la FPR sube de 0,239 a 0,414. Lo mismo ocurre con state = INT (FPR 0,547) y con las conexiones sin servicio identificado. Los segmentos con sttl = 31 o servicio dns no generan falsas alarmas. El solapamiento no explica el problema: la FPR es parecida en las filas de test ya vistas en train/val (0,208) y en las no vistas (0,192). **Conclusión:** el archivo de test tiene una distribución de tráfico normal distinta de la del archivo de entrenamiento (un cambio de distribución). La dependencia de sttl ya se veía en validación (92 % de las FP de v2, sección 2), pero la validación sale del mismo archivo que train y no podía anticipar que en test ese segmento pesaría el doble. Se reporta como limitación y no se corrige con el test. Una v3 futura debería validar con particiones que imiten ese cambio y revisar la dependencia de sttl.

6. Prueba de recuperación en entorno nuevo

scripts/prueba_recuperacion.sh se ejecutó en una carpeta nueva (~/recuperacion_20261028) con un entorno virtual nuevo, sin reutilizar cachés. Pasos: (1) clonar GitHub en el commit `3265bc1ef567` e instalar requirements.txt; (2) dvc pull desde DagsHub con caché vacía y verificar las huellas SHA-256 de los 2 CSV originales y de los 4 archivos preparados (todas OK); (3–4) con una caché de Hugging Face vacía, descargar v1 y v2 por su SHA completo, comprobar el hash de cada modelo y predecir las mismas 30 filas fijas de validación (estratificadas, semilla 2026); (5) comparar con las salidas guardadas antes de la recuperación. Tolerancia declarada: $|\Delta \text{ score}| \leq$ 1e-09. Resultado: **v1 y v2 con 30/30 etiquetas iguales y diferencia máxima 0 y 0: RESULTADO OK**. Duración: ~146 s. Entrada, salidas, hashes, log y comparación están en evidencias/recuperacion/. **Nota de acceso:** DagsHub exige iniciar sesión para leer el almacenamiento DVC aunque el repositorio sea público (el primer intento, sin credenciales, recibió HTTP 401). Por eso el revisor usa su propia cuenta gratuita y su token, guardados solo en .dvc/config.local. Nunca se comparten tokens.

7. Limitaciones

(1) UNSW-NB15 es un dataset de laboratorio de 2015: el resultado no demuestra eficacia en una red actual. (2) El test tiene un cambio de distribución en el tráfico normal (sección 5) y su FPR (0,192) está lejos del objetivo del 5 %; en producción, v2 generaría muchas más alertas de las previstas. (3) El 10,1 % del test repite entradas de train/val. (4) El umbral se elige y se reporta sobre la misma validación. (5) Variables como sttl pueden ser artefactos del laboratorio donde se generó el dataset más que señales generalizables. (6) model.joblib es un pickle: requiere scikit-learn 1.7.2 y solo debe cargarse de una fuente confiable. (7) Leer los datos requiere una cuenta de DagsHub.

Uso de IA. Claude (Anthropic), mediante Claude Code en VS Code, ayudó a estructurar el código, ejecutar los pasos, analizar los falsos positivos y redactar la documentación. La autora verificó las huellas, la ausencia de fuga, la recuperación y el recálculo de métricas (detalle en entrega.md).

Referencias

Moustafa, N. y Slay, J. (2015). UNSW-NB15: a comprehensive data set for network intrusion detection systems. Military Communications and Information Systems Conference (MilCIS), IEEE. https://research.unsw.edu.au/projects/unsw-nb15-dataset

Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. JMLR 12, 2825–2830. Errores comunes: https://scikit-learn.org/stable/common_pitfalls.html

DVC — Data Version Control. https://dvc.org/doc

MLflow Tracking. https://mlflow.org/docs/latest/tracking.html

Hugging Face Hub — descarga por revisión y model cards. https://huggingface.co/docs/hub

Mitchell, M. et al. (2019). Model Cards for Model Reporting. FAccT.

Zuluaga, J. (2026). Defiende el servidor, taller evaluable, MLOps 2026-II, Universidad Externado de Colombia.